

R2.01 - Développement Orienté Objets

A propos de Gradle

Arnaud Lanoix Brauer

Arnaud.Lanoix@univ-nantes.fr



Nantes Université

Département informatique

Compilation / exécution de `Hello.kt`

```
// 1er programme Kotlin

fun main() {
    println("*** Hello students !!!! ***)
}
```

Compilation / exécution de `Hello.kt`

```
// 1er programme Kotlin

fun main() {
    println("*** Hello students !!!! ***)
}
```

- **Compilation** via la commande `kotlinc` (terminal) :

```
kotlinc src/Hello.kt -d build
```

```
src/
+- Hello.kt
build/
```

Compilation / exécution de Hello.kt

```
// 1er programme Kotlin

fun main() {
    println("*** Hello students !!!! ***)
}
```

- **Compilation** via la commande `kotlinc` (terminal) :

```
kotlinc src/Hello.kt -d build
```

- Résultat de la compilation :

```
HelloKt.class dans bin/
```

```
src/
+- Hello.kt
build/
+- HelloKt.class
```

Compilation / exécution de Hello.kt

```
// 1er programme Kotlin

fun main() {
    println("*** Hello students !!!! ***)
}
```

- **Compilation** via la commande `kotlinc` (terminal) :

```
kotlinc src/Hello.kt -d build
```

- Résultat de la compilation :

```
HelloKt.class dans bin/
```

- **Exécution** via le lancement de la machine virtuelle Java :

```
kotlin -cp build HelloKt
```

(ou `java -cp build HelloKt`)

```
src/
+- Hello.kt
build/
+- HelloKt.class
```

Compilation / exécution de Hello.kt

```
// 1er programme Kotlin

fun main() {
    println("*** Hello students !!!! ***)
}
```

- **Compilation** via la commande `kotlinc` (terminal) :

```
kotlinc src/Hello.kt -d build
```

- Résultat de la compilation :

```
HelloKt.class dans bin/
```

- **Exécution** via le lancement de la machine virtuelle Java :

```
kotlin -cp build HelloKt
```

(ou `java -cp build HelloKt`)

- Affichage dans le terminal :

```
*** Hello students !!!! ***
```

```
src/
+- Hello.kt
build/
+- HelloKt.class
```

Des projets plus complexes... en réalité

Un projet "réel" comporte :

- des fichiers sources nombreux, organisés en différents dossiers
 - ▶ dans des langages différents
- des **librairies internes** et/ou **externes**
 - ▶ externe = téléchargée via un dépôt externe
- plusieurs "sous-projets" à compiler séparément, et/ou successivement
- plusieurs sous-projets" à exécuter séparément
- des **tests** à exécuter, concernant des "sous-projets" séparés
- de la **documentation** à générer
- une librairie à construire
- une **version** du ou des langages, du compilateur, de la JVM
- etc.

Exemple d'un projet (un peu) plus complexe

```
projet/  
  +-- README.md  
  +-- src/  
    |   +-- Main.kt  
    |   +-- AutreFichier.kt  
    |   +-- ...  
  +-- tests/  
    |   +-- QqTests.kt  
  +-- libs/  
    |   +-- une.lib.locale.1.2.3.jar  
  +-- libs_externes/  
    |   +-- junit.x.x.jar    // la lib pour les tests  
    |   +-- autre.lib.externe.z.y.x.jar  
  +-- build/  
    +-- classes/  
    +-- tests/
```


Exemple d'un projet (un peu) plus complexe

Compilation

```
kotlinc src/*  
-cp libs/une.lib.locale.1.2.3.jar:  
    libs_externes/autre.lib.externe.z.y.x.jar:  
    build/classes/  
-d build/classes/
```

```
projet/  
+-- README.md  
+-- src/  
|   +-- Main.kt  
|   +-- AutreFichier.kt  
|   +-- ...  
+-- tests/  
|   +-- QqTests.kt  
+-- libs/  
|   +-- une.lib.locale.1.2.3.jar  
+-- libs_externes/  
|   +-- junit.x.x.jar    // la lib pour les tests  
|   +-- autre.lib.externe.z.y.x.jar  
+-- build/  
    +-- classes/  
    +-- tests/
```

Exemple d'un projet (un peu) plus complexe (2)

```
projet/  
  +-- README.md  
  +-- src/  
  |   +-- ...  
  +-- tests/  
  |   +-- QqTests.kt  
  +-- libs/  
  |   +-- une.lib.locale.1.2.3.jar  
  +-- libs_externes/  
  |   +-- junit.x.x.jar    // la lib pour les tests  
  |   +-- autre.lib.externe.z.y.x.jar  
  +-- build/  
      +-- classes/  
      |   +-- MainKt.class  
      |   +-- AutreFichier.class
```

Exemple d'un projet (un peu) plus complexe (2)

Compilation des tests

```
kotlinc tests/QqTests.kt
  -cp libs_externes/junit.x.x.jar:
    libs/une.lib.locale.1.2.3.jar:
    libs_externes/autre.lib.externe.z.y.x.jar:
    build/classes/
  -d build/test
```

```
projet/
+-- README.md
+-- src/
|   +-- ...
+-- tests/
|   +-- QqTests.kt
+-- libs/
|   +-- une.lib.locale.1.2.3.jar
+-- libs_externes/
|   +-- junit.x.x.jar    // la lib pour les tests
|   +-- autre.lib.externe.z.y.x.jar
+-- build/
    +-- classes/
        |   +-- MainKt.class
        |   +-- AutreFichier.class
```

Exemple d'un projet (un peu) plus complexe (3)

```
projet/  
  +-- README.md  
  +-- ...  
  +-- libs_externes/  
    |   +-- junit.x.x.jar    // la lib pour les tests  
    |   +-- autre.lib.externe.z.y.x.jar  
  +-- build/  
    +-- classes/  
      |   +-- MainKt.class  
      |   +-- AutreFichier.class  
    +-- tests/  
      +-- QqTests.class
```

Exemple d'un projet (un peu) plus complexe (3)

Exécution des tests

```
java -jar libs_externes/junit.x.x.jar
--class-path build/classes:
           build/tests:
           libs/une.lib.locale.1.2.3.jar:
           libs_externes/autre.lib.externe.z.y.x.jar
--select-class QqTests
```

```
projet/
+-- README.md
+-- ...
+-- libs_externes/
|   +-- junit.x.x.jar    // la lib pour les tests
|   +-- autre.lib.externe.z.y.x.jar
+-- build/
    +-- classes/
    |   +-- MainKt.class
    |   +-- AutreFichier.class
    +-- tests/
        +-- QqTests.class
```

Gradle

Gradle est un **moteur de production** fonctionnant sur la plateforme Java.

- <https://gradle.org/>
- successeur de Makefile, Ant, Maven
- Compatible avec de nombreux langages de programmation : Java, C++, Scala, Groovy, Kotlin, ...
- Tâches de construction standard (compilation, tests, jars, documentation, etc.)
- Extension (langages, tâches, ...) via des plugins
- Possibilité de coder ces propres tâches

Structure standard d'un projet Gradle

```
projet/  
  +-- README.md  
  +-- gradle/  
    |   +-- wrapper/  
    |       +-- gradle-wrapper.jar  
  +-- gradlew  
  +-- gradlew.bat  
  +-- src/  
    |   +-- main/  
    |       |   +-- kotlin/  
    |       |       +-- Main.kt  
    |   +-- test/  
    |       +-- kotlin/  
    |           +-- QqTests.kt  
  +-- libs/  
    |   +-- une.lib.locale.1.2.3.jar  
  +-- build.gradle.kts  
  +-- gradle.properties  
  +-- settings.gradle.kts  
  +-- build/
```

Tâches gradle

- Compilation des fichiers sources de `src/main/kotlin/`

```
./gradlew classes
```

- Exécution de la méthode `main()`

```
./gradlew run
```

- Compilation des tests de

```
./gradlew testClasses
```

- Exécution des tests `src/test/kotlin/`

```
./gradlew test
```

- Création d'un .jar

```
./gradlew jar
```

- Effacement des fichiers de compilation

```
./gradlew clean
```

- ...

Extraits de build.gradle.kts

```
plugins {  
    kotlin("jvm") version "1.9.21"  
}  
repositories {  
    maven {  
        url = uri("http://nexus.dep-info.iut-nantes.univ-nantes.prive/reposit  
        isAllowInsecureProtocol = true  
    }  
}  
dependencies {  
    implementation(files("libs/info.but1.collections.nutarray-1.0.jar"))  
    testImplementation("org.junit.jupiter:junit-jupiter:5.11.2")  
}  
tasks.test {  
    useJUnitPlatform()  
}  
kotlin {  
    jvmToolchain(11)  
}
```